Ders 13 Genişletilmiş Notu: Seminer Sunumları: Dil Modeliyle Metin Sıkıştırma, DeepZip, Çok Katmanlı Görüntü Temsili, BWT-MTF-Aritmetik Kodlama ve Akıllı Sözlük Tabanlı Kodlama

Ders 13 Genişletilmiş Notu: Seminer Sunumları: Dil Modeliyle Metin Sıkıştırma, DeepZip, Çok Katmanlı Görüntü Temsili, BWT-MTF-Aritmetik Kodlama ve Akıllı Sözlük Tabanlı Kodlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yapısı
- 2. Sunum 1: Dil Modeliyle Metin Sıkıştırma
- 3. Sunum 2: DeepZip, Tekrarlayan Ağlarla Kayıpsız Sıkıştırma
- 4. Sunum 3: Çok Katmanlı Görüntü Temsili ve Görüntü Sıkıştırma
- 5. Sunum 4: BWT, Öne Taşıma ve Aritmetik Kodlama ile Metin Sıkıştırma
- 6. Sunum 5: Akıllı Sözlük Tabanlı Kodlama (IDBE)
- Örnek Sorular

1. Dersin Yapısı

Bu ders dönemin son kaydıdır ve önceki üç dersteki gibi öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırma ile ilgili yayımlanmış bir makaleyi özetledi; hoca sunumdan sonra yorum yaptı ve gerekirse soru sordu. Bu derste beş sunum yapıldı: dil modeliyle metin sıkıştırma, RNN tabanlı kayıpsız sıkıştırıcı DeepZip, çok katmanlı görüntü temsiliyle görüntü sıkıştırma, Burrows-Wheeler dönüşümü, öne taşıma (move-to-front) ve aritmetik kodlamayı birleştiren bir metin sıkıştırma çalışması ve akıllı sözlük tabanlı kodlama (IDBE).

Sunumların hepsi İngilizce yapıldı. Altyazı bu bölümlerde sık sık anlamsız tekrara düştü (örneğin 00:11:28-00:12:06, 00:16:06, 00:18:29-00:19:29, 00:43:59-00:57:45 arasının büyük kısmı, 01:10:19, 01:11:38, 01:22:39). Bu yüzden anlatım büyük ölçüde ekrandaki slaytlardan kuruldu; altyazı yalnız akışı tamamlamak için kullanıldı.

2. Sunum 1: Dil Modeliyle Metin Sıkıştırma

İlk sunumda bir öğrenci “Compression Through Language Modeling” başlıklı çalışmayı anlattı (ekrandan, 00:07:20). Çalışmanın fikri, doğal dil işlemede kullanılan n-gram dil modellerinin verdiği olasılıkları Huffman kodlamasının girdisi olarak kullanmak ve metni kelime düzeyinde sıkıştırmaktır.

2.1 Veri sıkıştırmanın temelleri

Sunan öğrenci önce genel tanımları hatırlattı. Veri sıkıştırmanın ilkesi veriyi olabildiğince az bitle saklamaktır; bu çoğu zaman yalnızca artıklığın (redundancy) atılması demektir. Özgün veri ile açılmış (decompress edilmiş) veri birebir aynıysa dönüşüm kayıpsız (lossless), farklıysa kayıplıdır (lossy). Metin sıkıştırmada kayıpsızlık esastır, çünkü tek bir karakterin değişmesi metnin anlamını bozabilir. Sunuma göre yaygın metin sıkıştırıcılarının bir kısmı Huffman kodlamasına dayanır; yöntem, kod tablosunu saklamak için sembol ve kodları kullanır.

2.2 Dil modeli ve n-gram

Dil modeli (language modeling, LM), belirli bir kelime dizisinin bir cümlede geçme olasılığını belirlemek için çeşitli istatistiksel ve olasılıksal tekniklerin kullanılmasıdır (ekrandan, 00:10:00). Dil modelleri kelime tahminlerine dayanak oluşturmak için metin derlemlerini analiz eder ve doğal dil işleme (NLP) uygulamalarında kullanılır. Sunuma göre bir dil modeli şu adımlarla çalışır:

1. Metin verisini analiz ederek kelime olasılıklarını belirler.
2. Bu veriyi, doğal dilde bağlam için kurallar koyan bir algoritmadan geçirerek yorumlar.
3. Öğrendiği kuralları kullanarak yeni cümleleri doğru tahmin eder ya da üretir.

Ardından üç kavram tanımlandı. Belge (document) bir gazete makalesi, roman, patent, bilimsel makale, blog yazısı, yorum ya da tweet olabilir. Derlem (corpus) belgelerin ya da çevrimiçi metin ve konuşma kayıtlarının bir topluluğudur. Treebank ise kelimeleri sözcük türü ve cümle yapısıyla etiketlenmiş (annotated) bir derlemdir.

Olasılık bir olayın ne kadar olası olduğunu 0 ile 1 arasında bir sayıyla gösterir; $P(A)$, A olayının olasılığıdır. $P(A \mid B)$ ise B'nin gerçekleştiği bilindiğinde A'nın olasılığıdır (koşullu olasılık).

N-gram, cümlelere ve kelime dizilerine olasılık atayan bir dil modelidir; n kelimelik bir diziyi ifade eder. Sunumdaki örnek "please turn your homework" cümlesidir:

- Unigram (tek kelime): "please", "turn".
- Bigram (iki kelime): "please turn", "turn your", "your homework".
- Trigram (üç kelime): "please turn your", "turn your homework".

N-gram modelleri hem bir n-gramın son kelimesinin önceki kelimeler verildiğinde olasılığını kestirmek hem de bütün bir kelime dizisine olasılık atamak için kullanılır.

2.3 Huffman kodlaması

Huffman kodlaması 1952'de David A. Huffman tarafından önerildi. Sunumda verilen adımlar şunlardır:

1. Bütün x sembollerinin sayıları toplanır.
2. En az sayıya sahip iki sembol x_1 ve x_2 bulunur.
3. Bu iki sembol yeni bir bileşik sembolde birleştirilir; bileşik sembol x_1'e bir dalla, x_2'ye öbür dalla (0 ve 1 etiketleriyle) bağlanır.
4. Birden çok sembol kaldıysa 2. adıma dönülür.
5. Aksi hâlde Huffman kodu hazırdır: her sembol, kökten o sembole inen bağlantıların etiket dizisiyle kodlanır.

Slaytta ayrıca entropi ve karmaşıklık (perplexity) denklemleri gösterildi (00:17:03), ama bu slaydın karesi alınmadı.

> **Ek bilgi (derste anlatılmadı):** Entropi $H = -\sum_i p_i \log_2 p_i$ ile, karmaşıklık genelde $PP = 2^H$ ile tanımlanır. Bir dil modelinin karmaşıklığı ne kadar düşükse sonraki kelimeyi o kadar iyi tahmin eder; bu da Huffman ya da aritmetik kodlamada daha kısa kodlar demektir.

2.4 Huffman ağacı örneği

Sunan öğrenci beş sembollü bir örnek çözdü (ekrandan, 00:17:35). Metin dosyasında "ABCDE" karakterlerinin görünme olasılıkları şöyle olsun: A = 0,25, B = 0,16, C = 0,22, D = 0,20, E =

0,17.

Ağaç şöyle kurulur (birleştirme sırası slayttaki ağaçtan okundu, ara adım eklendi):

1. En küçük iki olasılık E (0,17) ve B (0,16): birleşince EB = 0,33.
2. Kalanlar: A 0,25, C 0,22, D 0,20, EB 0,33. En küçük ikisi D ve C: CD = 0,42.
3. Kalanlar: A 0,25, EB 0,33, CD 0,42. En küçük ikisi A ve EB: EBA = 0,58.
4. EBA (0,58) ile CD (0,42) birleşir: kök = 1.

Slaytta sol dallar 0, sağ dallar 1 ile etiketlenmiştir. Buna göre kodlar: A = 01, B = 001, C = 10, D = 11, E = 000.

Ortalama kod uzunluğu:

$$\bar{L} = 2(0{,}25)+3(0{,}16)+2(0{,}22)+2(0{,}2)+3(0{,}17) = 0{,}50+0{,}48+0{,}44+0{,}40+0{,}51 = 2{,}33 \text{ bit/sembol}$$

Entropi:

$$H = -(0{,}25\log_2 0{,}25+0{,}16\log_2 0{,}16+0{,}22\log_2 0{,}22+0{,}2\log_2 0{,}2+0{,}17\log_2 0{,}17) \approx 2{,}3 \text{ bit/sembol}$$

İki değer python ile doğrulandı ($\bar{L} = 2{,}33$, $H = 2{,}3026$). Ortalama uzunluk entropinin çok az üzerindedir.

2.5 Sıkıştırma algoritması

Önerilen sıkıştırma algoritması beş adımdan oluşur (ekrandan, 00:18:15):

1. Unigram, bigram ve trigramlar için bir dil modeli eğitilir.
2. Her bigram için üçüncü kelimenin gözlenmiş herhangi bir kelime ya da UNK (bilinmeyen kelime simgesi) olma olasılığı bulunur.
3. Her bigram için, üçüncü kelimenin dağılımına göre bir Huffman ağacı kurulur.
4. Kelimelerin olasılıklarına göre bir unigram Huffman ağacı kurulur.
5. Kelimeler sırayla Huffman ağacındaki karşılık gelen bitlerle değiştirilir. Kelime bilinmiyorsa UNK kodlanır ve daha düşük dereceli modele geçilir; unigram modeli de kelimeyi bilmiyorsa karakter düzeyinde sıkıştırmaya geçilir.

Dosyanın sıkıştırılması (ekrandan, 00:20:00) aynı mantığı izler: girdi dosyasında ilerlenir ve her kelime, eğitim kümesinden kurulan Huffman ağaçları kullanılarak önündeki iki kelimeye göre kodlanır. Trigram bilinmiyorsa UNK kodlanır ve unigram modeline (o da Huffman ile kodlanmıştır) dönülür. Unigram da bilinmiyorsa eğitim kümesinde eğitilmiş karakter düzeyinde bir Huffman koduyla kodlanır. Açma işlemi aynı davranışın taklit edilmesiyle yapılır.

2.6 Uzantılar

Daha iyi sıkıştırma oranı için doğal dil işleme tekniklerine dayanan iki uzantı eklendi (ekrandan, 00:20:15):

- **Belirli bağlamlar (specific contexts):** Metin spor, siyaset, iş ya da başka bir konuya sınıflandırılır ve genel sözlük yerine o konuya özel sözlük (spor sözlüğü, siyaset sözlüğü, iş sözlüğü) kullanılır.
- **Kayan bağlam penceresi (sliding context window):** Dosya sıkıştırılırken pencereye giren kelimelerin sayıları artırılır, pencereden çıkanların sayıları azaltılır. Böylece trigram ve bigramlar açısından yerel bağlam daha iyi kullanılır ve daha temsilî ağırlıklar elde edilir.

2.7 Sonuçlar

Yöntem Treebank ve Blip derlemlerinden alınan cümle dosyalarında bzip2 ve gzip ile karşılaştırıldı (ekrandan, 00:21:38). Slaydın başlığı “Competitive with Gzip”tir. Boyutlar byte cinsindendir:

Dosya	Özgün	bzip2	gzip	Metin tabanlı
Treebank-sent.test.txt	558324	145853	198522	202186
Treebank-sent.validate.txt	554908	147401	199373	201645
Blip-sent.test.txt	5931314	1514353	2105318	2151026
Blip-sent.validate.txt	5957667	1519737	2112540	2134206

Python ile hesaplanan oranlara göre bzip2 dosyaları özgün boyutun yaklaşık %26'sına, gzip ve metin tabanlı yöntem yaklaşık %36'sına indirir. Metin tabanlı yöntemin çıktısı gzip'ten %1-2 büyüktür; yani gzip ile rekabet eder ama onu geçmez, bzip2'nin ise belirgin biçimde gerisindedir.

Sonuç bölümünde şu noktalar vurgulandı: belirli bir bigramdan sonra gelecek kelimelerin olasılığını kestirmek için bir dil modeli eğitmek ve bunun Huffman ağacını kurmak, bilinmeyen kelimeler için karakter tabanlı sıkıştırma uygulamak, çok basit bir teknikle kelimeleri sınıflandırırken yüksek doğruluk elde etmek ve ek olarak kayan pencereyi kullanmak.

Sunumdan sonra sınıftan soru gelmedi.

3. Sunum 2: DeepZip, Tekrarlayan Ağlarla Kayıpsız Sıkıştırma

İkinci sunumda bir öğrenci "DeepZip: Lossless Compression using Recurrent Networks" başlıklı çalışmayı anlattı (ekrandan, 00:24:15).

> **Ek bilgi (derste anlatılmadı):** DeepZip büyük olasılıkla Goyal ve arkadaşlarının 2018'de yayımladığı çalışmadır. Daha sonraki DZip ve benzeri sinir ağı tabanlı sıkıştırıcılar aynı "olasılık kestirici + aritmetik kodlayıcı" düzenini kullanır.

3.1 Motivasyon ve problem

Sunuma göre üretilen veri miktarında çok büyük bir artış vardır ve 3B sanal gerçeklik verisi, otonom sürüş haritalaması için nokta bulutu verisi gibi yeni veri türleri ortaya çıkmıştır. Bu veriler için genel amaçlı, iyi çalışan kayıpsız sıkıştırıcılara ihtiyaç vardır.

Aritmetik kodlama, bilinen bir dağılım için entropi sınırına ulaşmanın verimli bir yoludur. Dağılım bilinmediğinde aritmetik kodlamanın uyarlamalı (adaptive) türleri tasarlanmıştır; bunlar koşullu k-gram model dağılımını öğrenerek sıkıştırır.

Çalışmanın amacı şudur: LSTM ve GRU tabanlı modeller uzun vadeli bağımlılıkları yakalamada ve sonraki karakteri tahmin etmede çok iyidir; bu özellik kayıpsız sıkıştırmayı iyileştirmek için kullanılabilir.

3.2 DeepZip'in yapısı

DeepZip kayıpsız sıkıştırma yapar ve iki ana bloktan oluşur:

- **RNN tabanlı olasılık kestiricisi:** Sinir ağı tabanlı bu blok, geçmiş semboller verildiğinde bir sonraki sembolün koşullu olasılığını kestirir.
- **Aritmetik kodlayıcı:** Bu koşullu olasılığı optimal sıkıştırma ve açma için kullanır.

Sunuma göre elde edilen sıkıştırma boyutu (örnek başına kullanılan ortalama bit sayısı), RNN kestiricisinin normalize çapraz entropi kaybına çok yakındır. Yani kestirici ne kadar iyi tahmin ederse sıkıştırma o kadar iyi olur.

3.3 Model çerçevesi

Model çerçevesi dört ilkeye dayanır (ekrandan, 00:30:00):

- **Çıktı olarak dağılım:** RNN tabanlı model alfabe kümesi üzerinde bir olasılık dağılımı üretir; bu dağılım aritmetik kodlayıcının girdisidir.
- **Aritmetik kodlayıcı durumu:** Aritmetik kodlayıcı, aralık (range) durumunu tutan bir sonlu durum makinesi (FSM, finite state machine) gibi düşünülebilir; bu durum, RNN'deki gizli durum gibi bir sonraki adıma aktarılır.

- **Girdinin nedenselliği (causality):** RNN kestiricisi nedensel olmalıdır; yalnız önceden kodlanmış sembollere dayanan girdi özellikleri kullanabilir.
- **Ağırlık güncellemesi:** Ağırlık güncellemesi yapılıyorsa kodlayıcıda ve kod çözücüde birebir aynı şekilde yapılmalıdır.

Slayttaki şekil (Wikipedia kaynaklı) aritmetik kodlamayı gösterir: (0,6; 0,2; 0,1; 0,1) olasılıklı i.i.d. bir kaynakta (0; 2; 3) dizisi kodlanır. Adımlar şöyledir (ara adım eklendi):

1. Başlangıç aralığı $[0, 1)$. Sembollerin aralıkları: 0 → $[0; 0{,}6)$, 1 → $[0{,}6; 0{,}8)$, 2 → $[0{,}8; 0{,}9)$, 3 → $[0{,}9; 1)$. İlk sembol 0 olduğu için yeni aralık $[0; 0{,}6)$.
2. Bu aralık aynı oranlarla bölünür: 0 → $[0; 0{,}36)$, 1 → $[0{,}36; 0{,}48)$, 2 → $[0{,}48; 0{,}54)$, 3 → $[0{,}54; 0{,}6)$. İkinci sembol 2: yeni aralık $[0{,}48; 0{,}54)$.
3. Yeniden bölünür: $[0{,}48; 0{,}516)$, $[0{,}516; 0{,}528)$, $[0{,}528; 0{,}534)$, $[0{,}534; 0{,}54)$. Üçüncü sembol 3: son aralık $[0{,}534; 0{,}54)$.

Bu aralıktaki herhangi bir sayı (diziyi tek başına temsil eder) kodlanır. DeepZip'te fark, her adımdaki dağılımın sabit olmayıp RNN tarafından geçmişe göre üretilmesidir.

3.4 Kodlama ve kod çözme işlemi

Kodlamada ilk sembol S_0 için varsayılan (default) bir olasılık dağılımı kullanılır; böylece kod çözücü de ilk sembolü çözebilir. Sonra $S_0, S_1, S_2, ...$ sembolleri sırayla RNN'e girer; RNN her adımda bir sonraki sembolün dağılımını üretir ve aritmetik kodlayıcı bu olasılıkla sembolü kodlar. Sonunda sıkıştırılmış veri elde edilir. Kod çözücü aynı RNN ve aynı varsayılan dağılımla aynı adımları izleyerek sembolleri geri çözer.

3.5 Sentetik veriyle deneyler

Entropisi bilinen sentetik kaynaklar kullanıldı. Entropi, sıkıştırma oranı için bir alt sınırdır; böylece farklı modellerin başarımı bu ideal sınırla karşılaştırılabilir. Kullanılan sentetik veri setleri:

- **i.i.d. kaynaklar:** Çeşitli dağılımlarla bağımsız ve özdeş dağılımlı semboller.
- **Markov-k kaynakları:** k adım önceki sembollere bağlı, sıfır entropili (yani deterministik olarak tahmin edilebilir) kaynaklar. k büyüdükçe bağımlılık uzar ve öğrenmek zorlaşır.

İncelenen modeller DeepZip-ChRNN (karakter düzeyinde vanilla RNN), DeepZip-ChGRU (karakter düzeyinde GRU) ve DeepZip-Feat'tir (özellik tabanlı). Sunuma göre 128 hücreli vanilla RNN modeli (DeepZip-ChRNN) Markov-k kaynaklarında 30 zaman adımından uzaktaki bağımlılıkları öğrenemez ve Markov-35 için hiç sıkıştırma yapamaz. DeepZip-ChGRU daha uzun bağımlılıkları öğrenebilir.

Karşılaştırma tablosu (ekrandan, 00:37:35; her veri seti 100 MB):

Veri seti	DeepZip-ChGRU	DeepZip-Feat	gzip	Adaptive-AE
Markov-20	200 KB	200 KB	5 MB	2 MB
Markov-30	1 MB	400 KB	40 MB	87 MB
Markov-50	30 MB	500 KB	63 MB	100 MB
Markov-80	100 MB	2 MB	69 MB	100 MB
Markov-100	100 MB	38 MB	100 MB	100 MB
Markov-140	100 MB	100 MB	100 MB	100 MB

Slayttaki yorumlar: gzip Markov-80 kaynaklarını bir ölçüde sıkıştırabilir. DeepZip-Feat modeli girdi olarak yalnız son sembolü değil son 50 sembolü ve 4-gram bağlam sayılarını alır; Markov-140 gibi daha zor kaynaklarda bile bir yere kadar başarılıdır. DeepZip modelleri Markov-k kaynaklarında gzip ve uyarlamalı aritmetik kodlayıcıdan (Adaptive-AE) daha iyidir.

3.6 Gerçek veri setleriyle deneyler

Gerçek veri setleri şunlardır (ekrandan, 00:38:45): sıkıştırılması zor sözde rastgele sayı dizileri (PRNG), Hutter Prize veri seti (50.000 avroluk ödüllü yarışmanın Wikipedia metin verisi),

EnWiki-9 (daha büyük bir XML Wikipedia dökümü) ve bir kromozom dizisi.

Veri seti	Boyut	DeepZip-ChGRU	DeepZip-Feat	gzip	En iyi özel
PRNG-31-13	100 MB	32 MB	500 KB	95 MB	-
PRNG-52-21	100 MB	85 MB	1 MB	100 MB	-
PRNG-73-31	100 MB	100 MB	62 MB	100 MB	-
Hutter-Prize	100 MB	18 MB	16,5 MB	34 MB	15,9 MB (CMIX)
EnWiki-9	1 GB	178 MB	168 MB	330 MB	154 MB (CMIX)
Chromosome-1	240 MB	48 MB	42 MB	57 MB	49 MB (MF-Compress)

PRNG dizilerinde DeepZip-Feat dizinin üreteç yapısını öğrenerek çok büyük bir sıkıştırma sağlar; gzip neredeyse hiç sıkıştıramaz. Kromozom verisinde DeepZip-Feat özel sıkıştırıcı MF-Compress'ten bile iyidir. Metin verilerinde ise DeepZip gzip'ten çok daha iyidir ama en iyi özel sıkıştırıcı CMIX'in gerisinde kalır. Sunan öğrenci bunun olası nedenini şöyle açıkladı: metin verisinde çok uzun bağımlılıklar yoktur; bağımlılıklar genelde 100 karakter civarında ya da cümle içinde kalır.

Sonuç (ekrandan, 00:40:00): deneyler temelde iki modelle yapıldı, karakter düzeyinde GRU modeli ve özellik tabanlı GRU modeli. İki model de sözde rastgele sayı dizileri (PRNG) gibi zor kaynaklarda belirgin biçimde iyi sonuç verdi.

3.7 Hocanın sorusu ve yorumu

Sunum sırasında hoca, giriş slaytında 3B sanal gerçeklik verisi geçtiği için çalışmanın üç boyutlu sanal gerçeklik verisi üzerinde mi yapıldığını, kullanılan verinin görüntü mü metin mi olduğunu sordu (00:33:25-00:34:54). Sunan öğrenci verinin metin olduğunu söyledi. Sunumun sonunda hoca, başta çalışmayı yalnız üç boyutlu görüntülerde yapılan bir çalışma sandığını, oysa önerilen yöntemin çok farklı veri setlerinde (metinde, kromozom gibi biyoenformatik verisinde, Wikipedia verilerinde) denendiğini ve başarısının da gayet iyi olduğunu belirtti (00:40:41).

4. Sunum 3: Çok Katmanlı Görüntü Temsili ve Görüntü Sıkıştırma

Üçüncü sunumda bir öğrenci "Multilayered Image Representation: Application to Image Compression" başlıklı çalışmayı anlattı (ekrandan, 00:42:00). Altyazı bu sunumun büyük bölümünde bozuk olduğu için anlatım slaytlardan ve altyazının okunabilen kısımlarından kuruldu.

> **Ek bilgi (derste anlatılmadı):** Bu başlıktaki çalışma büyük olasılıkla Meyer, Averbuch ve Coifman'ın 2002 civarında IEEE Transactions on Image Processing'de yayımlanan makalesidir.

4.1 Çok katmanlı temsil fikri

Çok katmanlı temsil tekniğinde görüntü önce katmanların üst üste binmesi olarak ayrıştırılır: kenar (cartoon) katmanı, düzgün bölge katmanı ve doku (texture) katmanı. Her katman bağımsız olarak, farklı bir dönüşüm ya da tabanla ve farklı bir bit hızında kodlanır. Sıkıştırılmış katmanların herhangi bir birleşimi her zaman anlamlı bir görüntü olarak yeniden kurulabilir.

Bir görüntü iki katmanın toplamı olarak da yazılabilir: çizgi film (cartoon) görüntüsü ve doku haritası. Cartoon görüntü görüntüdeki belirgin parçaların ya da kenarların tanımını verir; doku haritası kenarlarla çevrili bölgelerin içini doldurur. Sunuma göre periyodik bir yapı içeren görüntüyü kodlamak için dalgacık (wavelet) yerine yerel kosinüs (local cosine) tabanı kullanmak avantaj sağlar. Çok katmanlı temsil fikri daha önce görüntüden gürültü gidermede başarıyla kullanılmıştır: görüntü önce dalgacıkla kodlanır, kalan hata (residual) da yine dalgacıkla kodlanır.

4.2 Algoritmanın ana döngüsü

Ana döngü şöyledir (ekrandan, 00:46:05): I görüntüsünün yaklaşıklamasının $N-1$. adıma kadar yapıldığını varsayalım. R_n, $n-1$. adımdaki yaklaşıklamanın kalan hatası olsun. $\hat{R}_n$'nin b_n bitle optimal yaklaşıklamasını veren en iyi taban $\{\psi_j^n,\ j \in E_n\}$, L_0 taban kütüphanesinden kurulur. Buna göre:

$$R^n = \hat{R}^n + R^{n+1}, \qquad \hat{R}^n = \sum_{j \in E_n} q_j^n \, \psi_j^n$$

Burada E_n en iyi tabanı oluşturan taban fonksiyonlarının indis kümesi, q_j^n ise nicelenmiş katsayılardır. Son olarak I'nın yaklaşıklaması, sıkıştırılmış kalan hataların toplamıyla kurulur: $\hat{R}^0 + \hat{R}^1 + \hat{R}^2 + ...$

Slayttaki ifadede taban gösterimi $\{\psi_j^1,\ j \in E_1\}$ olarak yazılmıştır; genel n. adım için E_n olması gerekir.

4.3 Örnek: üç yineleme

Bir bina fotoğrafı üzerindeki örnekte üç yineleme gösterildi (ekrandan, 00:50:00):

1. **Birinci yineleme:** Yoğunluğun parçalı düzgün değişimleri çok az sayıda dalgacık katsayısıyla kodlanır; sonuç $\hat{R}^0$'dır. Görüntü bulanık ama ana yapıları içerir.
2. **İkinci yineleme:** Kalan hata $R_1 = I - \hat{R}^0$ yerel kosinüs tabanıyla sıkıştırılır; bu doku katmanının sıkıştırılmasıdır. Çatı kiremitleri gibi dokular bu katmanda görünür.
3. **Üçüncü yineleme:** Doku da çıkarılmıştır; $R_2 = R_1 - \hat{R}^1$ rastgele gürültüye benzer.

4.4 Maliyet fonksiyonu ve niceleme

En iyi tabanı seçmek için kullanılan maliyet fonksiyonu, gerçek skaler niceleme ve entropi kodlamanın maliyetini yansıtır, ama hesaplaması çok daha hızlıdır. İki terimden oluşur: skaler niceleyicinin sıfır olmayan çıktı seviyelerinin işaretini ve büyüklüğünü kodlamanın maliyeti ve bu sıfır olmayan çıktı seviyelerinin konumunu kodlamanın maliyeti. Altyazıya göre Laplace dağılımına dayalı bir skaler niceleyici ve ardından entropi kodlama kullanılır.

4.5 Deneyler

Deneylerde yöntem, dalgacık kodlayıcı ve SPIHT ile karşılaştırıldı:

- **Bina görüntüsü:** Dalgacık kodlayıcı binaların çatısındaki kiremitleri koruyamadı (ekrandan, 00:53:12). Altyazıya göre bu örnekte çok katmanlı yöntem 0,125 bit/piksel hızında 25,23 dB PSNR verdi.
- **Barbara görüntüsü:** Masa örtüsündeki doku iyi korundu. Ancak yüzde, yüz ile atkı arasındaki sınırda kalan bloklardan kaynaklanan bazı desenler (artefaktlar) ortaya çıktı.
- **Palyaço (clown) görüntüsü:** Parçalı düzgün bölgelerle periyodik yapıların karışımını içerir ve dalgacık paketlerine (wavelet packets) uygundur; dalgacık paketleri ikinci katmanı kodlamak için optimal bir kütüphane sağlar. SPIHT duvar kâğıdının ve palyaçonun gömleğinin dokusunu koruyamadı; çok katmanlı algoritma görsel kalite bakımından daha iyi bir görüntü kurdu.

4.6 Hocanın yorumu

Hoca sunumdan sonra bu makalenin görüntü işleme üzerine, SPIHT algoritmasıyla ilgili bir çalışma olduğunu belirtti (00:58:18). Sınıfa görüntü işleme ile ilgili proje konusu alan olup olmadığını sordu; alan yoktu.

5. Sunum 4: BWT, Öne Taşıma ve Aritmetik Kodlama ile Metin Sıkıştırma

Dördüncü sunumda bir öğrenci, cep telefonu uygulamasında Burrows-Wheeler dönüşümü, öne taşıma (move-to-front, MTF) ve aritmetik kodlamayla metin sıkıştırmayı inceleyen bir makaleyi anlattı. Hoca konuyu "genel bir şey" olarak tanıttı.

5.1 Giriş ve yöntem

Sıkıştırma, verinin bellek kullanımını azaltmak ve iletim süresini kısaltmak için kod kümelerine dönüştürülmesidir; bu, veri kalitesi düşürülerek ya da düşürülmeden yapılabilir. Veri metin, resim, ses ya da video olabilir. Sıkıştırma algoritmaları kayıpsız ve kayıplı olarak iki sınıfa ayrılır. Metin dosyaları mobil depolama aygıtları dahil birçok depolama ortamında hâlâ baskındır. Makalenin vaka çalışması bir mobil uygulamadır.

Makalede üç algoritmanın dört birleşimi (şema) karşılaştırılır:

- **Şema I:** Yalnız aritmetik kodlama.
- **Şema II:** MTF + aritmetik kodlama.
- **Şema III:** BWT + aritmetik kodlama (MTF olmadan).
- **Şema IV:** BWT + MTF + aritmetik kodlama.

5.2 Aritmetik kodlama

Aritmetik kodlama, bir girdi sembolünü belirli bir kodla değiştirme fikrini atlar; girdi dosyasındaki sembol dizisini tek bir aritmetik sayıyla değiştirir. Girdi mesajı uzadıkça bu sayı için daha çok bit gerekir. Kodlama sonucu, 0'dan büyük ya da eşit ve 1'den küçük tek bir sayıdır; bu sayı tek anlamlı olarak çözülerek sembol dizisinin aynısı elde edilir. Çıktı sayısını kurmak için sembollere olasılıklar atanır; her karaktere $[0, 1)$ aralığının, görünme olasılığına karşılık gelen bir parçası verilir.

5.3 Öne taşıma (move-to-front) kodu

MTF, veriyi kendisi sıkıştırmayan ama artıklığı azaltmaya yardım eden bir dönüşüm algoritmasıdır. Ana fikir, verideki her sembolü yakın zamanda kullanılan semboller yığınındaki (listesindeki) indisiyle değiştirmektir. Aynı sembolün uzun tekrarları sıfırlarla, uzun süredir görülmeyen semboller ise büyük sayılarla değiştirilir. Sonunda veri bir tam sayı dizisine dönüşür; veride çok yerel ilişki (local correlation) varsa bu sayılar küçük olma eğilimindedir ve sonraki entropi kodlayıcı tarafından iyi sıkıştırılır.

5.4 Burrows-Wheeler dönüşümü

Burrows-Wheeler dönüşümü (BWT), blok sıralamalı ve kayıpsız bir dönüşümdür. Sunuma göre 1983'te bulunmuş ve 1994'te Michael Burrows ile David Wheeler tarafından teknik rapor olarak yayımlanmıştır. BWT bir metni genelde daha kolay sıkıştırılabilen yeni bir diziye dönüştürür. Dikkat çekici yanı yalnız daha kolay sıkıştırılabilir bir çıktı üretmesi değil, aynı zamanda tersinir olmasıdır.

Sunumdaki örnekte özgün dize "fathimahas"tır (ekrandan, 01:06:30). Üç adım uygulanır:

1. **Adım 1:** Dize birer karakter sola döndürülür (shift rotate); n karakterli bir dize için $n-1$ döndürme yapılır ve özgün satırla birlikte n satırlık bir matris elde edilir.
2. **Adım 2:** Matrisin bütün satırları artan sırada sıralanır. Sunan öğrenci, ilk karakterleri aynı olan satırlarda (örneğin "a" ile başlayan üç satır, "h" ile başlayan iki satır) sonraki karakterlere bakılarak sıranın belirlendiğini gösterdi.
3. **Adım 3:** BWT çıktısı sıralanmış listenin son sütunudur.

Slayttaki sonuç "aaafhhimst" olarak verildi (ekrandan, 01:10:00). Slayta göre dönüşümün faydası, dönüştürülmüş metnin sonraki sıkıştırma algoritmalarına daha uygun olmasıdır; BWT çıktısı MTF koduna girdi olur.

Sıralı döndürmeler (python ile hesaplandı, ara adım eklendi):

```
0  ahasfathim
1  asfathimah
2  athimahasf
3  fathimahas   <- özgün dize
4  hasfathima
5  himahasfat
6  imahasfath
7  mahasfathi
8  sfathimaha
9  thimahasfa
```

Ders sonrası not: Sunumda BWT çıktısı “aaafhhimst” olarak verildi. Bu, sıralı matrisin ilk sütunudur, yani dizenin karakterlerinin sıralanmış hâlidir. Tek başına tersinir değildir, çünkü aynı harflerden oluşan bütün dizeler aynı sonucu verir. Sunumda da söylendiği gibi BWT çıktısı son sütundur: yukarıdaki tablodan “mhfsathiaa” ve özgün dizenin satır indisi 3 (0’dan sayarak) elde edilir. Python ile doğrulandı.

5.5 MTF örneği

Sunumda MTF, “aaafhhimst” dizisine a’dan z’ye alfabe listesiyle uygulandı (ekrandan, 01:11:15). Her adımda karakterin listedeki indisi yazılır, karakter listenin başına taşınır:

Adım	Karakter	Adımdan önceki liste (baş kısmı)	Çıktı
1	a	a b c d e f …	0
2	a	a b c d e f …	0
3	a	a b c d e f …	0
4	f	a b c d e f …	5
5	h	f a b c d e g h …	7
6	h	h f a b c d e g …	0
7	i	h f a b c d e g i …	8
8	m	i h f a b c d e g j k l m …	12
9	s	m i h f a b c d e g j k l n o p q r s …	18
10	t	s m i h f a b c d e g j k l n o p q r t …	19

Sonuç: 0, 0, 0, 5, 7, 0, 8, 12, 18, 19 (python ile doğrulandı). Tekrar eden “aaa” ve “hh” gruplarının sıfırlara dönüştüğü görülür.

Ders sonrası not: Doğru BWT çıktısı “mhfsathiaa” kullanılsaydı MTF çıktısı 12, 8, 7, 18, 4, 19, 4, 11, 3, 0 olurdu (python ile doğrulandı).

5.6 Sıkıştırma oranı ve sonuçlar

Sıkıştırma oranı şöyle hesaplanır:

$$CR = \frac{\text{sıkıştırılmış dosya boyutu}}{\text{özgün dosya boyutu}} \times 100\%$$

Bu tanımla en iyi algoritma en küçük oranı verendir. Deney verisi, 32 kitaptan (bölümden) oluşan bir hadis metninin Endonezce çevirisinin metin dosyalarıdır; kitaplar dosya boyutu çeşitliliğini göstermek için seçilmiştir.

Şema IV’ün sonuçları (ekrandan, 01:15:05; blok boyutları byte):

Dosya	Boyut (byte)	1024	2048	4096	8192
Kitab Hibah.txt	5937	3747	3367	3213	2997
Kitab Kafarat.txt	12934	6810	6190	5697	5285
Kitab Hudud.txt	27851	14142	12786	11615	10838
Kitab Hukum.txt	59657	28973	25837	23441	21604

Dosya	Boyut (byte)	1024	2048	4096	8192
Kitab Thaharoh.txt	118991	55246	48413	43196	38814
Kitab Pelaksanaan	220133	102299	90238	80592	73065
Kitab 1-10.txt	618278	-	253907	226748	205996
Kitab 11-32.txt	183127	86947	77442	69765	63704
Kitab 1-32.txt	801405	-	330288	295426	-

En iyi oran Kitab Thaharoh'ta 8192 byte blokla elde edilir: $38814/118991 = \%32{,}62$. En kötü oran Kitab Hibah'ta 1024 byte blokla elde edilir: $3747/5937 = \%63{,}11$. İki oran python ile doğrulandı. Tabloda blok boyutu büyüdükçe her dosyada sıkıştırılmış boyut küçülür.

Sunan öğrencinin yorumuna göre BWT'nin girdi metnini sıralı karakter dizisine dönüştürüp ardından MTF ve aritmetik kodlamanın uygulanması sıkıştırma oranını en iyi sonuca (%32,62) taşır. Makalenin sonucu, BWT + MTF + aritmetik kodlama birleşiminin mobil telefon uygulamasında kullanılmasıdır.

5.7 Makale eleştirisi

Sunan öğrenci makaleyi birkaç kez okuduktan sonra bazı eleştiriler getirdi: makalenin 5. sayfasındaki 10. tablonun adı 9 olmalıdır, çünkü makalede 9. tablo yoktur; ayrıca 10. tabloda boş hücreler vardır ve yazarlar bu boşlukların nedenini açıklamamıştır, oysa bu hücrelerin sonuçları önemli olabilir. Şema IV tablosunda da (yukarıda "-" ile gösterilen) boş hücreler görülmektedir.

6. Sunum 5: Akıllı Sözlük Tabanlı Kodlama (IDBE)

Son sunumda bir öğrenci "An Intelligent Dictionary Based Encoding Algorithm for Text Data Compression for High Speed Data Transmission over Internet" başlıklı makaleyi anlattı.

6.1 Giriş

Sıkıştırma algoritmaları, veriyi saklamak için gereken alanı azaltmak üzere artık veriyi azaltır. Veri sıkıştırma, mevcut bant genişliğini iyi kullanarak iletişim maliyetini düşürmenin cazip bir yoludur. İnternet üzerinden iletilen sayısal veri (metin, fotoğraf, video, ses, programlar) son on yılda çok hızlı arttı ve bu eğilimin süreceği öngörülüyor. Bu yüzden veriyi olabildiğince küçültüp ağı en iyi biçimde kullanan algoritmalar geliştirmek anlamlıdır. Çalışmanın hedefi kayıpsız metin sıkıştırma problemidir.

Yazarlar, kaynak metne uygulanabilen ve mevcut (arka uç, backend) bir algoritmanın sıkıştırma yeteneğini artıran akıllı, tersinir dönüşümler geliştirdi (ekrandan, 01:20:00). Temel felsefe metni önişleyip daha verimli sıkıştırılabilen ara bir biçime çevirmek ve bunu yaparken dilin doğal artıklığından yararlanmaktır. Bunun için akıllı sözlük tabanlı kodlama (Intelligent Dictionary Based Encoding, IDBE) stratejisi önerildi. Metni klasik sıkıştırmadan önce önişlemenin sıkıştırma verimini çok daha iyi artırdığı gözlenmiştir.

Sıkıştırma yöntemleri iki sınıfa ayrılır. Sözlük yöntemleri birkaç sembolü tek bir kod sözcüğüyle temsil eder; sıkıştırma başarımı kod sözcüğünün uzunluğuna bağlıdır. Kelimeler ve metnin diğer parçaları sözlükteki bir girdinin indisiyle değiştirilir.

6.2 İlgili çalışmalar

İnternet, dijital kütüphaneler ve bilgi erişim sistemleri sayesinde son on yılda metin bilgisi olağanüstü arttı; sunuma göre metin verisi bütün internet trafiğinin yaklaşık %45'ini oluşturur. Kayıpsız metin sıkıştırma için Huffman, aritmetik kodlama, LZ ailesi ve Unix compress gibi birçok algoritma geliştirildi.

BWT bir veri bloğunu alıp bir sıralama algoritmasıyla yeniden düzenler. Çıktı bloğu girdiyle tamamen aynı elemanları içerir, yalnız sıraları farklıdır. Dönüşüm tersinirdir; özgün sıra kayıpsız geri kurulabilir. BWT bütün veri bloğuna bir kerede uygulanır. Çıktısı genelde önce MTF

aşamasından, sonra run-length kodlayıcıdan (RLE), en sonda da bir entropi kodlayıcıdan (genelde aritmetik ya da Huffman) geçirilir. Açma sırasında bu aşamalar ters sırayla uygulanır: aritmetik çözme, RLE çözme, MTF çözme ve ters BWT.

Star encoding: Girdi dosyalarında beklenen sık kelimelerden büyük bir sözlük kurulur. Sözlük önceden hazırlanmalı, sıkıştırıcı ve açıcı tarafından bilinmelidir. Sözlükteki her kelimenin, olabildiğince çok harfin yıldız (*) karakteriyle değiştirildiği bir eşdeğeri vardır; ideal olarak en sık kelimeler en yüksek yıldız oranına sahip olur. Sunumda Project Gutenberg'deki Romeo ve Juliet metninden bir parçanın özgün ve Star kodlanmış hâli gösterildi. Kodlanmış metin özgün metinle tamamen aynı sayıda karakter içerir ama yıldızlar baskındır. Star encoding'in sonraki bir değişikliği LIPT'tir (Length Index Preserving Transform); kökeni benzer birkaç dönüşüme dayanır.

6.3 Problem ve temel fikir

Makalede daha yüksek sıkıştırma oranı sağlayacak daha iyi bir kodlama yaklaşımı önerilir. Yazarlar fikri şu gözlemle açıklar: harfleri karıştırılmış ya da hatalı yazılmış bir cümleyi çoğu insan rahatça okuyabilir, çünkü görsel algı sistemimiz her kelimeyi tam harf dizisiyle değil, yaklaşık bir imza (signature) deseniyle tanır ve beynimizde her hatalı kelimeyi doğru kelimeyle eşleştiren bir sözlük vardır. Aynı biçimde, bilgisayar için kelimelerin imzaları benzersiz oldukları sürece keyfi seçilebilir.

Geliştirilen algoritma iki adımlıdır: önce akıllı bir sözlük kurulur, sonra girdi metni bu sözlükle kodlanır.

6.4 Sözlük kurma algoritması

Sözlük kurma algoritması (MakeDict) birden çok kaynak dosyayı girdi alır (ekrandan, 01:30:00):

1. Girdi dosyalarındaki bütün kelimeler çıkarılır.
2. Kelime tabloda varsa geçme sayısı bir artırılır; yoksa tabloya eklenir ve sayısı bire ayarlanır.
3. Tablo geçme sıklığına göre azalan sırada sıralanır.
4. Kodlar şöyle verilir:
   - İlk 218 kelimeye kod olarak ASCII 33-250 aralığındaki karakterler verilir.
   - Kalan kelimelerin her birine bu aralıktaki (33-250) iki ASCII karakterinin bir permütasyonu sırayla verilir. Hâlâ kelime kalırsa üç karakterlik, gerekirse dört karakterlik permütasyonlar verilir.
5. Yalnız kelimeleri ve kodlarını içeren yeni bir tablo oluşturulur ve sözlük olarak bir dosyada saklanır.
6. Dur.

ASCII 33-250 aralığında $250 - 33 + 1 = 218$ karakter vardır; bu yüzden tek karakterlik kod alabilecek kelime sayısı 218'dir. Böylece en sık kelimeler en kısa kodları alır.

6.5 Kodlama algoritması

Kodlama algoritması girdi dosyası `inp` ile başlar (ekrandan, 01:27:30):

```
A. Sözlüğü oku; bütün kelimeleri ve kodlarını bir tabloda sakla.
B. inp boş olmadığı sürece:
   1. inp'ten karakterleri oku ve token oluştur.
   2. Token 1 karakterden uzunsa:
      1. Token'ı tabloda ara.
      2. Bulunamazsa token'ı olduğu gibi çıktı dosyasına yaz.
         Bulunursa:
         1. Kelimenin kodunun uzunluğunu bul.
         2. Asıl kod = uzunluk + tablodaki kod. Uzunluk, kod çözmede
            işaret görevi görür ve ASCII 251-254 ile gösterilir
            (251 = 1 uzunluklu kod, 252 = 2 uzunluklu kod, ...).
         3. Asıl kodu çıktı dosyasına yaz.
```

```
         4. Sonraki karakteri oku; boşluksa yok say. Başka bir
            karakterse onu sonraki token'ın ilk karakteri yap ve
            boşluk olmadığını gösteren işaret karakterini (ASCII 255)
            ekledikten sonra B'ye dön.
   Değilse (token 1 karakterse):
      1. 1 karakterlik token'ı yaz.
      2. Karakter ASCII 251-255'ten biriyse, metnin parçası olduğunu
         ve işaret olmadığını göstermek için karakteri bir kez daha yaz.
C. Dur.
```

Algoritmaya göre kodlanmış bir kelimeden sonra gelen boşluk yazılmaz; boşluk yoksa bu durum ASCII 255 ile işaretlenir.

6.6 Sonuçlar

Örnek veriden, kodlanmış metnin klasik BWT tabanlı dönüşüm modeli için daha iyi sıkıştırılabilir bir biçim sağladığı görülür. Karakter başına bit (BPC) ve dönüşüm süresi üç durum için karşılaştırıldı: düz BWT, Star encoding ile BWT ve önerilen IDBE ile BWT. Canterbury derlemindeki sonuçlar (ekrandan, 01:31:55):

Dosya	Boyut (KB)	BWT BPC	BWT süre (s)	BWT+Star BPC	BWT+Star süre (s)	BWT+IDBE BPC	BWT+IDBE süre (s)
alice29.txt	148,5	2,45	3	2,39	6	2,11	4
Asyoulik.txt	122,2	2,72	2	2,61	7	2,32	4
cp.html	24,0	2,6	1	2,27	4	2,13	3
fields.c	10,9	2,35	0	2,20	4	2,06	3
grammar.lsp	3,60	2,88	0	2,67	4	2,44	3
kennedy.xls	1005,6	0,810	10	0,823	17	0,976	17
lcet10.txt	416,8	2,38	7	2,25	12	1,87	7
plrabn12.txt	470,6	2,80	10	2,69	13	2,30	8
ptt5	501,2	0,846	27	0,847	33	0,856	31
sum	37,3	2,80	2	2,75	4	2,89	4
xargs.1	4,1	3,51	1	3,32	4	2,93	2

Tablodan şu sonuçlar çıkar: metin içerikli dosyalarda (alice29, Asyoulik, lcet10, plrabn12, cp.html, fields.c, grammar.lsp, xargs.1) IDBE en düşük BPC'yi verir; örneğin lcet10.txt'de BPC 2,38'den 1,87'ye iner. kennedy.xls, ptt5 ve sum dosyalarında IDBE düz BWT'den kötü sonuç verir; sum'da en iyisi BWT + Star'dır. Süre bakımından IDBE, tablodaki her dosyada Star encoding'den hızlı ya da ona eşittir.

Sunumun son kısmına göre ideal bir kanalda iletim süresindeki azalma sıkıştırma miktarıyla doğru orantılıdır; ama dalgalanan bant genişliği, tıkanıklık ve paket anahtarlama protokolleri olan tipik bir internet ortamında bu tam olarak geçerli değildir. Sonuçlar metin sıkıştırmada mevcut yöntemlere göre belirgin bir iyileşme ve mevcut yöntemlere göre ek bir güvenlik düzeyi sağladığını göstermiştir.

Sunumdan sonra sınıftan soru gelmedi.

Örnek Sorular

Soru 1 (ekrandan, 00:17:35). Bir metin dosyasında A, B, C, D, E karakterlerinin olasılıkları sırasıyla 0,25, 0,16, 0,22, 0,20 ve 0,17'dir. Huffman ağacını kurun, kodları bulun; ortalama kod uzunluğunu ve entropiyi hesaplayın.

Çözüm (slaytta verildi): Birleştirmeler E + B = 0,33, C + D = 0,42, EB + A = 0,58, 0,58 + 0,42 = 1. Kodlar: A = 01, B = 001, C = 10, D = 11, E = 000. Ortalama uzunluk $2(0{,}25) + 3(0{,}16) + 2(0{,}22) + 2(0{,}2) + 3(0{,}17) = 2{,}33$ bit/sembol; entropi $\approx 2{,}3$ bit/sembol (python ile 2,3026).

Soru 2 (ekrandan, 00:30:00). (0,6; 0,2; 0,1; 0,1) olasılıklı i.i.d. bir kaynakta (0; 2; 3) dizisi aritmetik kodlamayla hangi aralığa kodlanır?

Çözüm (slaytta verildi): $[0,1) \rightarrow 0$ sembolüyle $[0;0{,}6) \rightarrow 2$ sembolüyle $[0{,}48;0{,}54) \rightarrow 3$ sembolüyle $[0{,}534;0{,}54)$. Bu aralıktaki bir sayı diziyi temsil eder.

Soru 3 (derste sözlü, 00:33:25). DeepZip çalışmasında kullanılan veri üç boyutlu sanal gerçeklik verisi mi, görüntü mü, metin mi?

Çözüm (derste verildi): Giriş slaytında 3B sanal gerçeklik verisi yalnız motivasyon olarak geçer. Yöntem metin (Hutter Prize, EnWiki-9), kromozom gibi biyoenformatik veri, sentetik Markov-k ve PRNG dizileri üzerinde denenmiştir. Hoca sunum sonunda yöntemin bu farklı veri setlerinde başarılı olduğunu belirtti.

Soru 4 (ekrandan, 01:06:30 ve 01:10:00). "fathimahas" dizesinin Burrows-Wheeler dönüşümünü bulun.

> **Çözüm (derste verilmedi):** Sunumda sonuç "aaafhhimst" olarak verildi; bu, sıralı matrisin ilk sütunudur. Doğru BWT, sıralı döndürmelerin son sütunudur: "mhfsathiaa", özgün dizenin sıralı listedeki indisi 3 (0'dan sayarak). Sıralı döndürmeler 5.4 bölümündeki tabloda verilmiştir; sonuç python ile doğrulandı.

Soru 5 (ekrandan, 01:11:15). "aaafhhimst" dizisine a'dan z'ye alfabe listesiyle öne taşıma (MTF) kodu uygulayın.

Çözüm (slaytta verildi): 0, 0, 0, 5, 7, 0, 8, 12, 18, 19. Her karakter listedeki indisiyle yazılır ve listenin başına taşınır; örneğin "f" ilk geldiğinde indisi 5'tir, ikinci "h" listenin başında olduğu için 0 olur. Python ile doğrulandı.

Soru 6 (ekrandan, 01:15:05). Şema IV'te 118991 byte'lık Kitab Thaharoh dosyası 8192 byte blokla 38814 byte'a, 5937 byte'lık Kitab Hibah dosyası 1024 byte blokla 3747 byte'a sıkıştırılıyor. Sıkıştırma oranlarını hesaplayın.

Çözüm (slaytta verildi): $CR = 38814/118991 \times 100\% = \%32{,}62$ (en iyi oran) ve $CR = 3747/5937 \times 100\% = \%63{,}11$ (en kötü oran). Python ile doğrulandı.

Soru 7 (ekrandan, 01:30:00). IDBE sözlüğünde ilk kaç kelime tek karakterlik kod alır ve neden?

Çözüm (slaytta verildi): İlk 218 kelime. Tek karakterlik kodlar ASCII 33-250 aralığından verilir ve bu aralıkta $250 - 33 + 1 = 218$ karakter vardır. ASCII 251-254 kod uzunluğunu, 255 boşluk yokluğunu işaretlemek için ayrılmıştır.

Dersin Toparlaması

Dönemin bu son dersinde beş seminer sunumuyla, derste görülen temel kodlama yöntemlerinin farklı makalelerde nasıl birleştirildiği görüldü. İlk iki sunum, iyi bir olasılık modelinin iyi bir sıkıştırma demek olduğunu iki ayrı yoldan gösterdi: birinci çalışmada n-gram dil modelinin olasılıkları her bigram için ayrı Huffman ağaçlarına dönüştürüldü ve gzip düzeyinde sonuç alındı; DeepZip'te ise RNN'in ürettiği koşullu dağılım doğrudan aritmetik kodlayıcıya verildi ve özellikle uzun bağımlılıklı sentetik kaynaklarda, PRNG dizilerinde ve kromozom verisinde gzip'in çok ötesine geçildi. Metinde bağımlılıklar kısa olduğu için en iyi özel sıkıştırıcı CMIX'in gerisinde kalındı.

Üçüncü sunum kayıplı tarafta görüntüyü dalgacıkla kodlanan düzgün katman, yerel kosinüs tabanıyla kodlanan doku katmanı ve gürültü olarak ayıran çok katmanlı bir temsili tanıttı; dokulu görüntülerde dalgacık kodlayıcıdan ve SPIHT'ten görsel olarak daha iyi sonuç verdi. Son iki sunum, kayıpsız metin sıkıştırmada dönüşüm zincirinin önemini öne çıkardı: BWT + MTF + aritmetik kodlama birleşimi tek başına aritmetik kodlamadan belirgin biçimde iyiydi (en iyi oran %32,62), IDBE ise sıklığa göre kurulan bir sözlükle metni BWT'den önce önişleyerek metin dosyalarında karakter başına biti düşürdü. BWT örneğinde çıktının sıralı matrisin ilk değil son sütunu olduğu da ayrıca not edildi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.